AI ASSISTED CODING
Assignment – 17.2

Name: Mohammed Rahil Raza

Ht.no: 2303A51433

bt.no: 21

Task Description – 1(Student Academic Data Cleaning)

• Task:

Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

Instructions:

• Handle missing values in marks, attendance, and department.

• Remove duplicate student records based on student ID.

• Standardize text fields such as department names.

• Encode categorical variables like gender and department.

Expected Output:

• A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.

Dataset file :

student.csv

```
student_id,name,gender,department,marks,attendance
101,Alice,Female,Computer Science,85,90
102,Bob,Male,computer science,78,85
103,Charlie,Male,Electrical,,88
104,David,Male,Mechanical,92,
105,Eva,Female,,75,80
```

Code screenshot:

```python
import pandas as pd

# Load dataset
data = pd.read_csv("student.csv")

print("Original Data:\n", data)

# ------------------------------
# 1. Handle Missing Values
# ------------------------------

# Fill missing marks with average marks
data['marks'] = data['marks'].fillna(data['marks'].mean())

# Fill missing attendance with average attendance
data['attendance'] = data['attendance'].fillna(data['attendance'].mean())

# Fill missing department with 'Unknown'
data['department'] = data['department'].fillna('Unknown')

# ------------------------------
# 2. Remove Duplicate Records
# ------------------------------

data = data.drop_duplicates(subset='student_id')

# ------------------------------
# 3. Standardize Text Fields
# ------------------------------

# Convert department names to lowercase then capitalize properly
data['department'] = data['department'].str.strip().str.lower()

# Standardize common department names
data['department'] = data['department'].replace({
    'computer science': 'Computer Science',
    'electrical': 'Electrical',
    'mechanical': 'Mechanical',
    'civil': 'Civil',
    'unknown': 'Unknown'
})

# ------------------------------
# 4. Encode Categorical Variables
# ------------------------------

# Encode gender (Male=0, Female=1)
data['gender'] = data['gender'].map({'Male': 0, 'Female': 1})

# One-hot encoding for department
data = pd.get_dummies(data, columns=['department'])
```

Output screenshot:

```
PS C:\Users\phani\OneDrive\Desktop\Btech\3-2\Ai19.2> python cleaned_students.py
Original Data:
    student_id     name  gender        department  marks  attendance
0         101    Alice  Female  Computer Science   85.0        90.0
1         102      Bob    Male  computer science   78.0        85.0
2         103  Charlie    Male        Electrical    NaN        88.0
3         104    David    Male        Mechanical   92.0         NaN
4         105      Eva  Female               NaN   75.0        80.0

Cleaned Data:
    student_id     name  gender  ...  department_Electrical  department_Mechanical  department_Unknown
0         101    Alice       1  ...                  False                  False               False
1         102      Bob       0  ...                  False                  False               False
2         103  Charlie       0  ...                   True                  False               False
3         104    David       0  ...                  False                   True               False
4         105      Eva       1  ...                  False                  False                True

[5 rows x 9 columns]

Cleaned dataset saved as 'cleaned_students.csv'
```

Task Description – 2 (Financial Transaction Data Preprocessing)

• Task:

Preprocess financial transaction data using AI-generated Python code.

Instructions:

• Convert transaction date columns to datetime format.

- Create derived features such as transaction month and year.
- Normalize the transaction amount column.
- Identify and handle extreme transaction values.

Expected Output:

- A transformed dataset with normalized values and newly generated time-based features.

Dataset file:

transactions.csv

```
transaction_id,user_id,transaction_date,amount,transaction_type
1,1001,2024-01-15,5000,Debit
2,1002,2024/02/20,12000,Credit
3,1003,15-03-2024,300,Debit
4,1004,2024-04-10,999999,Credit
5,1005,,4500,Debit
```

Code screenshot:

eprocess_transactions.py > ...

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import numpy as np
# Load dataset
transactions = pd.read_csv("transactions.csv")
print("Original Data:\n", transactions)
# -----------------------------------
# 1. Convert transaction_date to datetime
# -----------------------------------

transactions['transaction_date'] = pd.to_datetime(
    transactions['transaction_date'],
    errors='coerce'
)
# Fill missing dates with forward fill
transactions['transaction_date'] = transactions['transaction_date'].fillna(method='ffill')

# -----------------------------------
# 2. Create Derived Features
# -----------------------------------

transactions['transaction_year'] = transactions['transaction_date'].dt.year
transactions['transaction_month'] = transactions['transaction_date'].dt.month

# -----------------------------------
# 3. Handle Extreme Values (Outliers)
# -----------------------------------

# Remove negative transactions (optional rule)
transactions['amount'] = transactions['amount'].apply(lambda x: abs(x))
# Cap extreme values using IQR method
Q1 = transactions['amount'].quantile(0.25)
Q3 = transactions['amount'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

transactions['amount'] = np.where(
    transactions['amount'] > upper_bound,
    upper_bound,
    transactions['amount']
)

transactions['amount'] = np.where(
    transactions['amount'] < lower_bound,
    lower_bound,
    transactions['amount']
```

```python
)

# ------------------------------------
# 4. Normalize Transaction Amount
# ------------------------------------

scaler = MinMaxScaler()
transactions['normalized_amount'] = scaler.fit_transform(transactions[['amount']])

# ------------------------------------
# Final Output
# ------------------------------------

print("\nProcessed Data:\n", transactions)

# Save cleaned dataset
transactions.to_csv("processed_transactions.csv", index=False)

print("\nProcessed dataset saved as 'processed_transactions.csv'")
```

Output screenshot:

```
PS C:\Users\phani\OneDrive\Desktop\Btech\3-2\Ai19.2> python preprocess_transactions.py
Original Data:
   transaction_id  user_id transaction_date  amount transaction_type
0               1     1001       2024-01-15    5000           Debit
1               2     1002       2024/02/20   12000          Credit
2               3     1003       15-03-2024     300           Debit
3               4     1004       2024-04-10  999999          Credit
4               5     1005              NaN    4500           Debit

Processed Data:
   transaction_id  user_id transaction_date  ...  transaction_year transaction_month  normalized_amount
0               1     1001       2024-01-15  ...              2024                 1           0.204793
1               2     1002       2024-01-15  ...              2024                 1           0.509804
2               3     1003       2024-01-15  ...              2024                 1           0.000000
3               4     1004       2024-04-10  ...              2024                 4           1.000000
4               5     1005       2024-04-10  ...              2024                 4           0.183007

[5 rows x 8 columns]
```

Task 3 – (Environmental Sensor Data Preparation)

• Task:

Clean and preprocess environmental sensor data using AI-assistedscripts.

Instructions:

• Handle missing values in temperature, humidity, and air_quality_index.

• Scale numerical features using standard scaling.

• Encode categorical fields such as sensor_status.

• Split the dataset into training and testing subsets.

Dataset file:

sensor_data.csv

```csv
sensor_id,temperature,humidity,air_quality_index,sensor_status
1,30.5,65,120,Active
2,28.0,70,,Inactive
3,,60,90,Active
4,35.2,,150,Faulty
5,29.8,75,110,Active
```

Code screenshot:’

preprocess_sensor_data.py > ...

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("sensor_data.csv")

print("Original Data:\n", data)

# ----------------------------------
# 1. Handle Missing Values
# ----------------------------------

# Fill missing numerical values with mean
data['temperature'] = data['temperature'].fillna(data['temperature'].mean())
data['humidity'] = data['humidity'].fillna(data['humidity'].mean())
data['air_quality_index'] = data['air_quality_index'].fillna(data['air_quality_index'].mean())

# ----------------------------------
# 2. Encode Categorical Variables
# ----------------------------------

# Convert sensor_status to numeric (One-hot encoding)
data = pd.get_dummies(data, columns=['sensor_status'])

# ----------------------------------
# 3. Scale Numerical Features
# ----------------------------------

scaler = StandardScaler()

numerical_cols = ['temperature', 'humidity', 'air_quality_index']
data[numerical_cols] = scaler.fit_transform(data[numerical_cols])

# ----------------------------------
# 4. Split Dataset
# ----------------------------------

X = data.drop('sensor_id', axis=1)
y = data['sensor_id']  # just for example (usually target variable)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------------
# Final Output
# ----------------------------------

print("\nProcessed Data:\n", data)
print("\nTraining Set:\n", X_train)
print("\nTesting Set:\n", X_test)
```

Output:

```
Original Data:
   sensor_id  temperature  humidity  air_quality_index sensor_status
0          1         30.5      65.0              120.0        Active
1          2         28.0      70.0                NaN      Inactive
2          3          NaN      60.0               90.0        Active
3          4         35.2       NaN              150.0        Faulty
4          5         29.8      75.0              110.0        Active

Processed Data:
   sensor_id  temperature  humidity  air_quality_index  sensor_status_Active  sensor_status_Faulty  sensor_status_Inactive
0          1    -0.157715      -0.5           0.129099                  True                 False                   False
1          2    -1.209147       0.5           0.000000                 False                 False                    True
2          3     0.000000      -1.5          -1.420094                  True                 False                   False
3          4     1.818978       0.0           1.678293                 False                  True                   False
4          5    -0.452116       1.5          -0.387298                  True                 False                   False

Training Set:
   temperature  humidity  air_quality_index  sensor_status_Active  sensor_status_Faulty  sensor_status_Inactive
4    -0.452116       1.5          -0.387298                  True                 False                   False
2     0.000000      -1.5          -1.420094                  True                 False                   False
0    -0.157715      -0.5           0.129099                  True                 False                   False
3     1.818978       0.0           1.678293                 False                  True                   False

Testing Set:
   temperature  humidity  air_quality_index  sensor_status_Active  sensor_status_Faulty  sensor_status_Inactive
1    -1.209147       0.5                0.0                 False                 False                    True

Processed dataset saved as 'processed_sensor_data.csv'
```

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

• Scenario:

A smart city system collects traffic data from multiple sensors,

resulting in noisy and inconsistent records.

Requirements:

• Standardize road and location names.

• Fill missing traffic density values using appropriate statistical

methods.

• Remove duplicate sensor readings.

• Normalize speed and vehicle count metrics.

• Generate a brief summary comparing dataset quality before and

after cleaning

Sample Code:

```
import pandas as pd

traffic = pd.read_csv("traffic_data.csv")

print(traffic.describe())
```

Expected Output:

• A cleaned and normalized traffic dataset with an accompanying data-quality improvement summary.

Dataset file:

traffic_data.csv

```
sensor_id,road_name,location,traffic_density,speed,vehicle_count
1,MG Road,Hyderabad,80,45,200
2,mg road,hyderabad,,50,220
3,Ring Road,Delhi,120,60,300
4,Ring road,delhi,110,,280
5,Highway 1,Mumbai,300,80,500
```

Code screenshot:

clean_traffic_data.py > ...

```python
import pandas as pd

# Load dataset
traffic = pd.read_csv("traffic_data.csv")

print("Before Cleaning:\n")
print(traffic.describe(include='all'))

# ------------------------------------
# 1. Standardize Text Fields
# ------------------------------------

traffic['road_name'] = traffic['road_name'].str.strip().str.lower().str.title()
traffic['location'] = traffic['location'].str.strip().str.lower().str.title()

# ------------------------------------
# 2. Handle Missing Values
# ------------------------------------

# Fill traffic_density with median (better for skewed data)
traffic['traffic_density'] = traffic['traffic_density'].fillna(
    traffic['traffic_density'].median()
)

# Fill speed and vehicle_count with mean
traffic['speed'] = traffic['speed'].fillna(traffic['speed'].mean())
traffic['vehicle_count'] = traffic['vehicle_count'].fillna(traffic['vehicle_count'].mean())

# ------------------------------------
# 3. Remove Duplicate Records
# ------------------------------------

traffic = traffic.drop_duplicates()
# ------------------------------------
# 4. Normalize Numerical Features
# ------------------------------------

for col in ['speed', 'vehicle_count']:
    min_val = traffic[col].min()
    max_val = traffic[col].max()
    traffic[col] = (traffic[col] - min_val) / (max_val - min_val)

# ------------------------------------
# 5. After Cleaning Summary
# ------------------------------------

print("\nAfter Cleaning:\n")
print(traffic.describe(include='all'))

# ------------------------------------
# 6. Save Cleaned Data
# ------------------------------------
traffic.to_csv("cleaned_traffic_data.csv", index=False)
print("\nCleaned dataset saved as 'cleaned_traffic_data.csv'")
```

Output:

```
PS C:\Users\phani\OneDrive\Desktop\Btech\3-2\Ai19.2> & C:/Users/phani/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/phani/OneDrive/Desktop/Btech/3-2/Ai19.2/clean_traffic_data.p
Before Cleaning:

        sensor_id road_name   location  traffic_density     speed  vehicle_count
count    5.000000        5          5         4.000000   4.00000       5.000000
unique        NaN        5          5              NaN       NaN            NaN
top           NaN  MG Road  Hyderabad              NaN       NaN            NaN
freq          NaN        1          1              NaN       NaN            NaN
mean     3.000000      NaN        NaN       152.500000  58.75000     300.000000
std      1.581139      NaN        NaN        99.791449  15.47848     119.163753
min      1.000000      NaN        NaN        80.000000  45.00000     200.000000
25%      2.000000      NaN        NaN       102.500000  48.75000     220.000000
50%      3.000000      NaN        NaN       115.000000  55.00000     280.000000
75%      4.000000      NaN        NaN       165.000000  65.00000     300.000000
max      5.000000      NaN        NaN       300.000000  80.00000     500.000000

After Cleaning:

        sensor_id road_name   location  traffic_density     speed  vehicle_count
count    5.000000        5          5         5.000000  5.000000       5.000000
unique        NaN        3          3              NaN       NaN            NaN
top           NaN  Mg Road  Hyderabad              NaN       NaN            NaN
freq          NaN        2          2              NaN       NaN            NaN
mean     3.000000      NaN        NaN       145.000000  0.392857       0.333333
std      1.581139      NaN        NaN        88.034084  0.382993       0.397213
min      1.000000      NaN        NaN        80.000000  0.000000       0.000000
25%      2.000000      NaN        NaN       110.000000  0.142857       0.066667
50%      3.000000      NaN        NaN       115.000000  0.392857       0.266667
75%      4.000000      NaN        NaN       120.000000  0.428571       0.333333
max      5.000000      NaN        NaN       300.000000  1.000000       1.000000

Cleaned dataset saved as 'cleaned_traffic_data.csv'
```

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

• Task:

Use AI-assisted Python scripting to clean and preprocess social

media text data for sentiment analysis.

Instructions:

• Remove duplicate posts and null text entries.

• Clean text by removing URLs, special characters, and extra spaces.

• Convert text to lowercase and perform basic tokenization.

• Remove stopwords and perform stemming or lemmatization.

• Generate basic text features such as word count and sentimentscore.

Sample Code:

```
import pandas as pd

import re

posts = pd.read_csv("social_media_posts.csv")

print(posts.head())
```

Expected Output:

A cleaned text dataset with extracted features suitable for sentiment analysis models.

Dataset file:

social_media_post.csv

```
post_id,text
1,I love this product! 😍 http://example.com
2,This is terrible... worst experience ever!!!
3,Amazing service and fast delivery :)
4,Check this out https://test.com great stuff
5,
6,I LOVE THIS PRODUCT! 😍 http://example.com
7,Not good, not bad, just okay.
8,Absolutely fantastic!!! #happy
9,Worst app ever!!! 😡😡
10,Good value for money.
```

Code screenshot:

preprocess_social_text.py > ...

```python
import pandas as pd
import re
from collections import Counter

# Load dataset
posts = pd.read_csv("social_media_posts.csv")

print("Original Data:\n", posts)

# ------------------------------------
# 1. Remove Duplicates & Nulls
# ------------------------------------

posts = posts.drop_duplicates(subset='text')
posts = posts.dropna(subset=['text'])

# ------------------------------------
# 2. Text Cleaning Function
# ------------------------------------

def clean_text(text):
    text = re.sub(r"http\S+", "", text)          # remove URLs
    text = re.sub(r"[^a-zA-Z\s]", "", text)    # remove special characters
    text = re.sub(r"\s+", " ", text)            # remove extra spaces
    return text.strip().lower()                 # lowercase

posts['clean_text'] = posts['text'].apply(clean_text)

# ------------------------------------
# 3. Tokenization
# ------------------------------------

posts['tokens'] = posts['clean_text'].apply(lambda x: x.split())

# ------------------------------------
# 4. Remove Stopwords
# ------------------------------------

stopwords = {
    'is', 'this', 'and', 'the', 'for', 'to', 'a',
    'of', 'in', 'on', 'not', 'just', 'out'
}

posts['tokens'] = posts['tokens'].apply(
    lambda words: [w for w in words if w not in stopwords]
)

# ------------------------------------
# 5. Simple Stemming (Manual)
# ------------------------------------

def simple_stem(word):
    for suffix in ['ing', 'ly', 'ed', 's']:
        if word.endswith(suffix):
            return word[:-len(suffix)]
```

preprocess_social_text.py > ...

```python
def simple_stem(word):
    for suffix in ['ing', 'ly', 'ed', 's']:
        if word.endswith(suffix):
            return word[:-len(suffix)]
    return word

posts['tokens'] = posts['tokens'].apply(
    lambda words: [simple_stem(w) for w in words]
)

# ------------------------------------
# 6. Feature Extraction
# ------------------------------------

# Word count
posts['word_count'] = posts['tokens'].apply(len)

# Simple Sentiment Score (basic rule-based)
positive_words = {'love', 'amazing', 'fantastic', 'good', 'great', 'happy'}
negative_words = {'worst', 'bad', 'terrible'}

def sentiment_score(words):
    score = 0
    for w in words:
        if w in positive_words:
            score += 1
        elif w in negative_words:
            score -= 1
    return score

posts['sentiment_score'] = posts['tokens'].apply(sentiment_score)

# ------------------------------------
# Final Output
# ------------------------------------

print("\nProcessed Data:\n", posts[['clean_text', 'tokens', 'word_count', 'sentiment_score']])

# Save cleaned dataset
posts.to_csv("processed_social_media_posts.csv", index=False)

print("\nProcessed dataset saved as 'processed_social_media_posts.csv'")
```

Output:

```
Original Data:
    post_id                                       text
0         1     I love this product! 😍 http://example.com
1         2  This is terrible... worst experience ever!!!
2         3          Amazing service and fast delivery :)
3         4   Check this out https://test.com great stuff
4         5                                          NaN

Processed Data:

                            clean_text                             tokens  word_count  sentiment_score
0                    i love this product                  [i, love, product]           3                1
1  this is terrible worst experience ever  [terrible, worst, experience, ever]           4               -2
2       amazing service and fast delivery      [amaz, service, fast, delivery]           4                0
3             check this out great stuff                [check, great, stuff]           3                1
5                    i love this product                  [i, love, product]           3                1
6             absolutely fantastic happy         [absolute, fantastic, happy]           3                2
7                         worst app ever                   [worst, app, ever]           3               -1
8                   good value for money                 [good, value, money]           3                1

✅ File saved as 'processed_social_media_posts.csv'
```